someone else will change it while you are reading it, and without having to set a lock.

Writing remains a bit more complicated[32]: in the functional world, a method like `set` would return not a boolean, but a new instance of the cache itself, where its content has been updated.

7.8 Cache Applications

This section would definitely be shorter if we list examples of systems that do *not* use cache!

Cache is ubiquitous, at all level, from processors to the highest-level application you can imagine.

Low level, hardware caches works slightly differently, though, so we better restrict to software cache.

As mentioned, many single- and multi-thread applications employ ad-hoc in-memory caches to avoid repeating expensive computations.

This kind of caches are usually objects (in OO[33] languages), provided by libraries, and allocating dynamic memory in the heap of the same application that uses them. In case of multi-thread applications, the cache might run in its own thread, and be shared among several threads (as we saw in the previous section, this configuration requires special attention to prevent race conditions).